



AI & LLM Foundation

AICB-P1 · Ngày 1 · Nền tảng

Phạm Mạnh

VinUniversity · Phase 1 · Tuần 1 · 15/03/2026

HÃY SUY NGHĨ...

*“Bạn đang dùng AI mỗi ngày –
nhưng thực sự bên trong nó làm gì?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Bức tranh AI tính đến 2026 và sau đó
2. LLM – Trái tim của AI hiện đại
3. Token Economy
4. Gọi API lần đầu
5. Thực hành

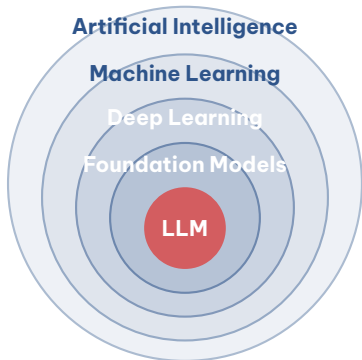
Bức tranh AI tính đến 2026 và sau đó

Từ Machine Learning đến Generative AI, Voice AI và Agentic AI

Sau buổi học này, bạn sẽ:

1. Đọc được bản đồ ứng dụng GenAI hiện nay: chatbot, voicebot, ảnh, video, code
2. Giải thích được Agentic AI và 3 khái niệm lõi: **agent harness, agent skills, tool/MCP**
3. Giải thích được cách LLM hoạt động (Transformer, token, next-token prediction)
4. Tính được chi phí API call dựa trên token economy
5. Gọi được API từ 3 providers: Anthropic, OpenAI, Google Gemini
6. Xây dựng chatbot đơn giản có streaming

Python 3.10+, VS Code/Cursor, API key (Anthropic hoặc OpenAI)



- **AI:** Máy thực hiện tác vụ “thông minh”
- **ML:** Học từ dữ liệu, không cần lập trình tường minh
- **DL:** Neural networks nhiều tầng
- **Foundation Models:** Mô hình nền tảng huấn luyện trên dữ liệu khổng lồ, áp dụng nhiều tác vụ – không chỉ text mà cả **ảnh, âm thanh, video**
- **LLM:** Foundation Model chuyên ngôn ngữ – nền tảng của GenAI và Agentic AI

Khóa học này tập trung vào LLM → xây dựng

Discriminative AI

Chức năng: Phân loại, dự đoán

Ví dụ:

- Spam filter
- Image classifier
- Fraud detection
- Chấm điểm tín dụng

Input → Label

Generative AI

Chức năng: Sinh nội dung mới

Ví dụ:

- Text: ChatGPT, Claude
- Voice: ElevenLabs, callbot
- Ảnh: Midjourney, Flux
- Video: Sora, Veo, Kling
- Code: Copilot, Cursor

Prompt → Content

Agentic AI

Chức năng: Tự lập kế hoạch & hành động

Ví dụ:

- AI coding agents
- Auto customer support
- Research agents
- Agent xử lý hồ sơ, đối soát

Goal → Plan → Action

LLM là **engine chung** cho cả Generative AI lẫn Agentic AI

Hành trình khóa học: LLM Foundation → Agent → Multi-Agent → Deploy → Evaluate

Modality	Công cụ tiêu biểu	Ứng dụng thực tế
Text / Chat	ChatGPT, Claude, Gemini	Chatbot CSKH, trợ lý nội bộ tra cứu quy trình, tóm tắt hợp đồng
Voice / Speech	Whisper, ElevenLabs, Realtime API, Gemini Live	Callbot tổng đài, nhắc lịch hẹn, khảo sát tự động, trợ lý giọng nói
Ảnh	Midjourney, Flux, Stable Diffusion, Nano Banana	Ảnh sản phẩm thương mại điện tử, banner marketing, thử đồ ảo
Video	Sora, Veo, Kling, Runway	TVC ngắn, video đào tạo, minh họa nội dung mạng xã hội
Code	Claude Code, Cursor, Copilot	Sinh code, refactor, viết test, migrate hệ thống cũ
Đa phương thức	GPT/Claude/Gemini vision, OCR	Đọc hóa đơn, CCCD, biểu mẫu; kiểm tra ảnh hiện trường bảo hiểm

Tên sản phẩm chỉ mang tính minh họa – thị trường thay đổi rất nhanh, hãy kiểm tra lại khi triển khai.

Chatbot (text)

Kênh: web, app, Zalo OA, Messenger, Slack-/Teams

Ví dụ đang chạy thật:

- Ngân hàng: tra cứu sao kê, hướng dẫn mở thẻ
- E-commerce: tư vấn sản phẩm, tra cứu đơn hàng
- Nội bộ: hỏi đáp quy trình, chính sách nhân sự

Điểm mạnh: rẻ, dễ log, dễ kiểm soát, gửi được link/ảnh/nút bấm

Ràng buộc: người dùng phải gõ; độ trễ vài giây vẫn chấp nhận được

Voicebot (giọng nói)

Kênh: tổng đài, hotline, app có mic, thiết bị/xe

Ví dụ đang chạy thật:

- Callbot nhắc lịch hẹn, nhắc đóng phí
- Tổng đài tự động thay IVR bấm phím
- Khảo sát hài lòng sau cuộc gọi
- Trợ lý giọng nói trong xe, trong nhà máy (rảnh tay)

Điểm mạnh: tiếp cận cả người không quen dùng app; xử lý song song hàng nghìn cuộc gọi

Ràng buộc: **độ trễ là sống còn** – trên 1 giây là hội thoại “gượng”

Cascaded pipeline (STT → LLM → TTS)

Ghép 3 model rời nhau:

1. STT (Speech-to-Text): Whisper, Deepgram, hoặc engine tiếng Việt nội địa

2. LLM: xử lý ý định, gọi tool, tra CSDL

3. TTS (Text-to-Speech): ElevenLabs, Cartesia, giọng đọc tiếng Việt

Ưu: kiểm soát từng khâu, log được transcript, dễ thay thế nhà cung cấp, rẻ hơn

Nhược: độ trễ cộng dồn (thường 1–2s), mất ngữ điệu và cảm xúc của người nói

Speech-to-speech (realtime)

Một model nhận thẳng audio và trả thẳng audio:

- OpenAI Realtime API
- Google Gemini Live API

Ưu: độ trễ thấp (~300–800ms), giữ được ngữ điệu, ngắt lời (barge-in) tự nhiên

Nhược: khó chèn logic nghiệp vụ giữa chừng, transcript kém tin cậy hơn cho audit, chi phí cao hơn, ít lựa chọn nhà cung cấp

Thực tế: nhiều hệ thống dùng lai – realtime cho hội thoại, pipeline cho luồng cần audit

Ngân sách độ trễ cho một lượt nói (mục tiêu < 1s):

Phát hiện dứt câu (endpointing)	200–400 ms
STT	100–300 ms
LLM (token đầu tiên)	300–600 ms
TTS (âm thanh đầu tiên)	100–300 ms

Tổng	0.7–1.6 s
-------------	------------------

Mẹo: stream mọi khâu – đừng chờ LLM sinh xong cả câu trả lời rồi mới đọc.

Các vấn đề đặc thù:

- **Barge-in:** người dùng ngắt lời → phải dừng TTS ngay
- **Endpointing:** biết khi nào người ta nói xong, không cướp lời
- **WER tiếng Việt:** giọng vùng miền, tên riêng, số tài khoản, đơn vị tiền
- **Đọc số:** “1.500.000” phải đọc thành “một triệu năm trăm nghìn”
- **Nhiều & đường truyền:** audio tổng đài 8kHz khác hẳn mic điện thoại
- **Chuyển người thật:** luôn cần đường thoát khi bot bí

Voicebot không phải “chatbot gắn thêm loa”. Ràng buộc thời gian thực buộc bạn thiết kế lại cả prompt (câu trả lời phải ngắn), cả luồng tool call, cả cách xử lý lỗi.

Sinh & sửa ảnh

- Ảnh sản phẩm: 1 ảnh gốc → 20 bối cảnh
 - Banner, thumbnail theo chiến dịch
 - Xóa/thay nền, thử đồ ảo
 - Minh họa cho tài liệu đào tạo
- Midjourney, Flux, SDXL, Nano Banana, Firefly*

Sinh video & avatar

- Video ngắn quảng cáo, social
 - Người dẫn AI cho video đào tạo
 - Lồng tiếng & dịch đa ngôn ngữ giữ giọng gốc
- Sora, Veo, Kling, Runway, Hey-Gen*
- Cảnh báo:** deepfake – cần quy tắc sử dụng nội bộ

Hiểu đa phương thức

Đây mới là phần tạo ROI nhanh nhất:

- Đọc hóa đơn, CCCD, biểu mẫu viết tay
- Kiểm tra ảnh hiện trường (bảo hiểm, giao hàng)
- Đọc bản vẽ, biểu đồ trong tài liệu kỹ thuật
- Kiểm duyệt nội dung ảnh

Vision của GPT / Claude / Gemini

Cùng một pattern API: gửi **prompt + dữ liệu** → nhận **nội dung**. Khác nhau chủ yếu ở modality, chi phí và độ trễ.

Agent – LLM biết hành động

Agent = LLM đặt trong một **vòng lặp**, được cấp **công cụ** và **quyền hành động**, tự quyết định các bước để đạt mục tiêu – thay vì chỉ trả lời một câu hỏi.



Chatbot dừng ở một lượt hỏi–đáp. Agent lặp cho tới khi xong việc – và vì thế nó cần một bộ khung để chạy an toàn.

Harness là toàn bộ phần *không phải model* bao quanh LLM. Cùng một model, harness tốt hay tệ tạo ra khác biệt rất lớn.

Harness gồm những gì:

- **Vòng lặp agent:** gọi model → chạy tool → đưa kết quả về → lặp
- **Quản lý ngữ cảnh:** nhồi gì vào context, khi nào nén (compaction)
- **Tool registry:** khai báo tool, validate tham số, xử lý lỗi
- **Quyền & sandbox:** được gọi API nào, lúc nào phải hỏi người
- **Bộ nhớ & phiên:** giữ trạng thái giữa các lượt
- **Subagent:** tách việc phụ để không làm bẩn context chính
- **Quan sát được:** log, trace, chi phí, điểm dừng

Harness có sẵn: Claude Agent SDK / Claude Code, OpenAI Agents SDK, LangGraph (Ngày 8).

Vì sao quan trọng:

- Model mạnh + harness kém = agent lan man, đốt token, không dừng
- Phần lớn lỗi “agent ngu” thực ra là lỗi harness: context tràn, tool trả lỗi khó hiểu, thiếu điều kiện dừng
- Đổi model mới chỉ mất vài dòng; đổi harness là viết lại hệ thống

Ngày 3–8: tự xây một harness, rồi chuyển sang LangGraph.

Agent Skill là một thư mục chứa hướng dẫn + script + tài liệu, mô tả cách làm *một loại việc cụ thể*. Agent chỉ nạp skill khi thật sự cần – gọi là **progressive disclosure**.

```
bao-cai-tai-chinh/  
  SKILL.md    # hướng dẫn chính  
  mau-bao-cai-xlsx  
  quy-tac-ke-toan.md  
  scripts/doi-soat.py  
  
# đầu file SKILL.md  
---  
name: bao-cai-tai-chinh  
description: Lập báo cáo tài  
             chính quý theo mẫu công ty  
---
```

Agent chỉ đọc dòng description của mọi skill. Khi việc khớp, nó mới mở toàn bộ nội dung skill đó ra.

Vì sao cần skills:

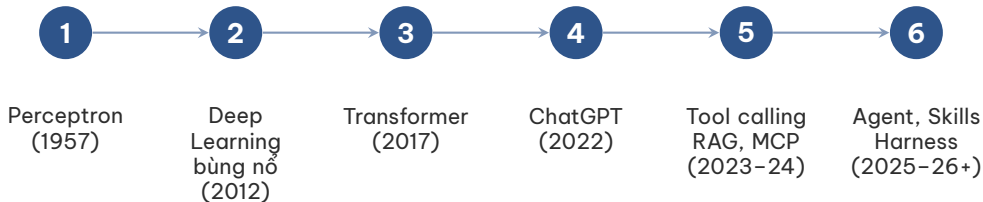
- Context window có hạn – không thể nhét toàn bộ quy trình công ty vào system prompt
- Know-how nội bộ (mẫu biểu, quy tắc, giọng văn thương hiệu) không nằm trong dữ liệu huấn luyện
- Viết bằng Markdown → người làm nghiệp vụ sửa được, không cần lập trình viên
- Đóng gói được, chia sẻ được giữa các team và các agent

Ví dụ skill thực tế:

- *Lập báo cáo quý*: mẫu Excel + quy tắc hạch toán
- *Viết theo brand voice*: giọng điệu + ví dụ tốt/xấu
- *Xử lý hồ sơ vay*: checklist giấy tờ + ngưỡng cảnh báo
- *Review code*: chuẩn code nội bộ + lỗi hay gặp

Khái niệm	Là gì	Trả lời câu hỏi	Ví dụ
Tool calling	Hàm agent gọi được, có schema tham số	<i>Agent làm được gì?</i>	<code>tra_cuu_don_hang(id)</code>
MCP	Chuẩn kết nối agent với tool/dữ liệu bên ngoài	<i>Cắm vào hệ thống nào?</i>	MCP server cho CRM, Jira, database
Agent Skills	Thư mục hướng dẫn + tài nguyên, nạp khi cần	<i>Làm việc đó thế nào cho đúng?</i>	Quy trình lập báo cáo quý
Harness	Vòng lặp, ngữ cảnh, quyền, log bao quanh model	<i>Chạy an toàn và bền ra sao?</i>	Claude Agent SDK, LangGraph

Bốn khái niệm này bổ sung cho nhau, không thay thế nhau. Chi tiết ở Ngày 4, 8, 9.



Mỗi bước không thay thế bước trước – nó chồng thêm một lớp năng lực lên trên.

78%

Doanh nghiệp
dùng AI

\$15.7T

GDP toàn cầu
từ AI (2030)

3.7x

ROI trung bình
trên mỗi \$1 đầu tư

Giai đoạn 2023–24 là “AI trả lời hay”. Từ 2025–26 trở đi, câu hỏi của doanh nghiệp đã đổi: AI có **tự làm xong việc** không, có **cắm được vào hệ thống** sẵn có không, và có **đo được ROI** không. Đó chính là lý do Agentic AI – và bộ ba harness, tool/MCP, skills – trở thành trọng tâm.

LLM – Trái tim của AI hiện đại

Transformer, Token, và cách LLM “suy nghĩ”

Large Language Model (LLM)

Mô hình ngôn ngữ lớn dựa trên kiến trúc **Transformer**, được huấn luyện trên lượng dữ liệu văn bản khổng lồ (hàng nghìn tỷ token). LLM có khả năng sinh văn bản, trả lời câu hỏi, viết code, và thực hiện reasoning phức tạp.

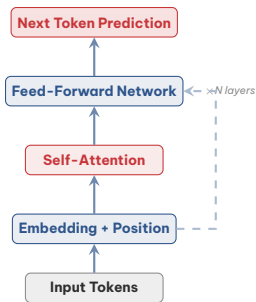
Đặc điểm chính:

- Decoder-only Transformer architecture
- Self-supervised pre-training + RLHF fine-tuning
- Next-token prediction – dự đoán từ tiếp theo
- Emergent capabilities xuất hiện khi scale lên

Năm	Cột mốc	Ý tưởng mới	Hạn chế còn lại
1958	Perceptron (Rosenblatt)	Nơ-ron nhân tạo học được từ dữ liệu	Chỉ phân tách tuyến tính, một lớp
1986	MLP + Backpropagation (Rumelhart et al.)	Nhiều lớp ẩn → học quan hệ phi tuyến	Đầu vào cố định, không hiểu thứ tự
1990	RNN (Elman)	Trạng thái ẩn quay vòng = bộ nhớ cho chuỗi	Vanishing gradient → quên ngữ cảnh xa
1997	LSTM (Hochreiter & Schmidhuber)	Cell state + cổng quên/vào/ra giữ phụ thuộc dài	Chạy tuần tự từng token → chậm
2013	Word2Vec (Mikolov)	Từ → vector, ngữ nghĩa thành phép toán	Một từ một vector, bất kể ngữ cảnh
2014	Seq2Seq + GRU (Sutskever; Cho)	Encoder-decoder dịch máy; GRU gọn hơn LSTM	Nén cả câu vào một vector → hụt ở câu dài
2014	Attention (Bahdanau et al.)	Decoder “nhìn lại” mọi vị trí đầu vào	Vẫn gắn với RNN nên vẫn tuần tự
2017	Transformer (Vaswani et al.)	Chỉ cần self-attention, bỏ recurrent → song song hóa	Tốn dữ liệu, GPU; chi phí $O(n^2)$
2018	BERT (encoder) & GPT-1 (decoder)	Pre-train quy mô lớn rồi fine-tune	BERT không sinh văn bản; GPT-1 còn nhỏ
2020–22	GPT-3 → ChatGPT	Scale + instruction tuning/RLHF	Ảo giác, thiếu dữ liệu mới → cần tool & agent

Mỗi cột mốc gỡ đúng một hạn chế của cột mốc trước – LLM là điểm cuối của chuỗi đó, không phải một phát minh đơn lẻ.

Nguồn: bài báo gốc của từng cột mốc – liệt kê ở slide Tài liệu tham khảo.



- **Self-Attention:** Mỗi token “nhìn” tất cả token khác trong context
- **Multi-Head:** Nhiều “góc nhìn” song song
- **Feed-Forward:** Xử lý phi tuyến từng vị trí
- **Residual connections:** Gradient chảy dễ dàng

Hai kiến trúc chính: Encoder-Decoder (BERT, T5) hiểu ngữ cảnh 2 chiều để phân loại, dịch thuật. **Decoder-only** (GPT, Claude, Gemini) đọc trái→phải để dự đoán token tiếp và sinh văn bản. *Ngày nay Decoder-only thắng thế nhờ scale tốt hơn.*

Attention – Cơ chế cho phép model “chú ý” đến các phần quan trọng nhất của input khi xử lý mỗi token

Ví dụ:

“Con **mèo** ngồi trên bàn. **Nó** rất đáng yêu.”

Khi xử lý “Nó”, attention sẽ gán trọng số cao cho
“mèo” – hiểu “Nó” chỉ con mèo, không phải cái bàn.

Công thức:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Q = Query, K = Key, V = Value

d_k = dimensionality (để normalize)

Token – Đơn vị nhỏ nhất mà LLM xử lý
– khoảng 0.75 từ tiếng Anh, 0.5 từ tiếng Việt

Tokenization: Tách text thành subword units

"Hello world" → 2 tokens

"Xin chào" → 3–4 tokens

"anthropic" → 3 tokens

"def func():" → 4 tokens

```
[ ``Tôì'' ] [ ``yêu'' ]
```

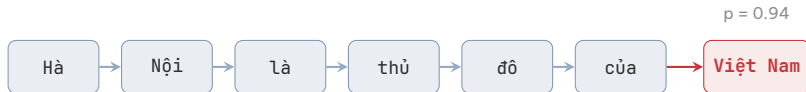
```
[ ``Việt'' ] [ ``Nam'' ]
```

→ 4+ tokens (mỗi từ có dấu = 1–2 tokens)

So sánh: "I love Vietnam" → 3 tokens

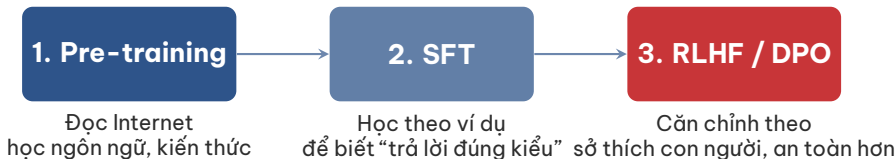
Thử: platform.openai.com/tokenizer

Lưu ý: Tiếng Việt tốn nhiều token hơn tiếng Anh (dấu, ký tự Unicode)
→ chi phí API cao hơn cho cùng nội dung.



- LLM **không “hiểu”** ngôn ngữ – nó dự đoán token có xác suất cao nhất
- **Temperature**: Điều chỉnh độ “sáng tạo” (0 = deterministic, 1 = random hơn)
- **Autoregressive**: Output token trở thành input cho bước tiếp theo

Lưu ý: LLM có thể tự tin đưa ra thông tin sai (hallucination) vì nó tối ưu xác suất, không phải sự thật.



Lưu ý: Analogy dễ nhớ: Pre-training = “đọc rất nhiều”, SFT = “được chỉ cách trả lời”, RLHF/DPO = “được uốn nắn để cư xử đúng hơn”.

Knowledge cutoff

Model không biết những gì xảy ra sau thời điểm training nếu không được cấp thêm dữ liệu/tools.

Hallucination

Model có thể trả lời rất tự tin nhưng sai vì đang tối ưu xác suất token, không phải tính đúng-sai.

Context window

Model chỉ “nhìn” được lượng token hữu hạn trong mỗi lần gọi. Quá dài thì tốn chi phí, và thông tin giữa prompt dễ bị quên.

Analogy

LLM giống một “học giả đọc rất nhiều” nhưng sống trong một bong bóng thời gian và chỉ được nhìn một số trang trước mặt.

Gợi ý dạy: nhấn mạnh rằng các giới hạn này giải thích vì sao sau này cần prompt tốt, context management, RAG và tools.

Token Economy

Chi phí, tốc độ và cách tính giá API

03



- Giá tính theo **1 triệu tokens** (1M tokens)
- Output tokens **đắt hơn** input tokens (3–5x)
- Giá giảm ~10x mỗi năm (GPT-4 level: \$20/M → \$2/M trong 2 năm)

Model	In	Out	Ctx	Loại	Khi nên dùng
Claude Opus 4.6	\$5.0	\$25	1M	Closed	Reasoning, code khó
Claude Sonnet 4	\$3.0	\$15	1M	Closed	Balanced choice
Claude Haiku 4.5	\$0.8	\$4	200K	Closed	Fast, cheap, routing
GPT-4o	\$5.0	\$20	128K	Closed	Multimodal, ecosystem
Gemini 2.5 Pro	\$1.25	\$10	1M	Closed	Long-context tasks
Llama 4 Scout	Free	Free	1M	Open	Self-host, private data

Lưu ý: Closed = API hosted; Open = self-host / control nhiều hơn. Giá tham khảo tháng 3/2026.

Nếu ưu tiên cost/latency

- FAQ, phân loại, trích xuất đơn giản
- Batch jobs số lượng lớn
- Trả lời ngắn, ít reasoning

Gợi ý: Haiku, Gemini Flash, model nhỏ/open-source

Nếu ưu tiên quality/reasoning

- Phân tích nhiều bước, code, planning
- Tài liệu dài, ngữ cảnh phức tạp
- Bài toán cần độ tin cậy cao

Gợi ý: Sonnet, Opus, GPT-4o, Gemini Pro

Lưu ý: Rule of thumb: bắt đầu từ model **đủ tốt và đủ rẻ**. Chỉ nâng model khi chất lượng thực sự chặn use case.

Prompt ví dụ: “Tóm tắt báo cáo tài chính Q1 trong 3 bullet và nêu 1 rủi ro chính.”

Claude

Mạch lạc, thiên về cấu trúc.

Phong cách: cẩn thận, “consulting style”.

GPT-4o

Ngắn gọn, tự nhiên, linh hoạt.

Phong cách: hợp app/chat, đa dụng.

Gemini

Mạnh khi context dài, nhiều tài liệu.

Phong cách: hợp workflow nhiều file.

*Gợi ý dạy: chạy live cùng 1 prompt để học viên thấy model selection không chỉ là “giá”, mà còn là **phong cách** + **độ phù hợp task**.*

Context Window – Số token tối đa mà LLM xử lý trong 1 lần gọi API (input + output)

- **128K tokens** $\approx$ 1 cuốn sách 300 trang
- **1M tokens** $\approx$ 4–5 cuốn sách
- Context càng dài $\rightarrow$ chi phí càng cao
- Thông tin ở giữa context dễ bị “quên” (Lost in the Middle)

Gemini 2.5	1M
Claude Sonnet 4	1M
Claude Opus 4.6	1M
GPT-4o	128K
Llama 4	1M

Scenario: Chatbot hỗ trợ khách hàng, 1000 lượt/ngày

Input trung bình: 500 tokens/lượt (câu hỏi + context)

Output trung bình: 200 tokens/lượt (câu trả lời)

Dùng Claude Sonnet 4:

Input: $500K \times \$3/1M = \1.50

Output: $200K \times \$15/1M = \3.00

Tổng/ngày: \$4.50

Tổng/tháng: ~\$135

Dùng Claude Haiku 4.5:

Input: $500K \times \$0.80/1M = \0.40

Output: $200K \times \$4/1M = \0.80

Tổng/ngày: \$1.20

Tổng/tháng: ~\$36

Lưu ý: Chọn model phù hợp: Haiku cho tác vụ đơn giản, Sonnet/Opus cho reasoning phức tạp.

Gọi API lần đầu

Từ “Hello World” đến production-ready call

04



- **Prompt:** system + user input + context
- **API Call:** gửi request tới model provider
- **Token Stream:** model sinh output từng chunk
- **Response:** text hoàn chỉnh + usage + stop reason

Lưu ý: Tư duy đúng cho PM/engineer: mỗi API call luôn có 3 thứ cần kiểm soát cùng lúc: **quality, latency, cost.**

- ☒ Python 3.10+ đã cài đặt
- ☒ VS Code hoặc Cursor IDE
- ☒ Tài khoản Anthropic API (có credit)
- ☒ Biến môi trường: ANTHROPIC_API_KEY
- ☐ OpenAI API key (không bắt buộc, dùng trong Lab)
- ☐ Google AI Studio key (không bắt buộc, dùng trong Lab)

```
import anthropic

# Reads API key from env var ANTHROPIC_API_KEY
client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {"role": "user",
         "content": "Explain what LLM is in 2 sentences"}
    ]
)

print(message.content[0].text)
```

Request (gửi đi):

- `model`: Model nào sử dụng
- `max_tokens`: Giới hạn output (bắt buộc Anthropic)
- `messages`: Lịch sử hội thoại
- `system`: System prompt (tùy chọn)

Response (nhận về):

- `content`: Nội dung trả lời
- `model`: Model thực tế được dùng
- `usage.input_tokens`: Số input tokens
- `usage.output_tokens`: Số output tokens
- `stop_reason`: `end_turn` | `max_tokens` | `tool_use`

temperature	Độ “sáng tạo” (0–1)	0 = deterministic; 1 = diverse. Dùng thấp cho code/phân tích, cao hơn cho sáng tạo
top_p	Nucleus sampling (0–1)	Chỉ chọn từ top tokens chiếm $p\%$ xác suất. Thường dùng 0.9–0.95
stop_sequences	Dừng ở chuỗi chỉ định	Hữu ích khi cần output có cấu trúc cố định hoặc cắt đúng điểm mong muốn

Bắt đầu với temperature=0 cho tác vụ cần ổn định. Chỉ tăng sampling khi thật sự cần đa dạng câu trả lời.


```
# === ANTHROPIC (Claude) ===
import anthropic
client = anthropic.Anthropic()
resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    messages=[{"role": "user", "content": "Hello"}]
)
print(resp.content[0].text)      # .content[0].text

# === OPENAI (GPT) ===
from openai import OpenAI
client = OpenAI()
resp = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello"}]
)
print(resp.choices[0].message.content)  # .choices[0]....
```

```
# === GOOGLE GEMINI ===
import google.generativeai as genai
model = genai.GenerativeModel("gemini-2.5-flash")
resp = model.generate_content("Hello")
print(resp.text)           # .text (simple!)
```

Tóm tắt khác biệt chính:

	Anthropic	OpenAI	Google
API call	messages.create()	chat.completions.create()	generate_content()
max_tokens	Required	Optional	Optional
Response	.content[0].text	.choices[0]...content	.text

```
import anthropic
client = anthropic.Anthropic()

# Streaming: receive response chunk by chunk
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Write a poem"}]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

# Get token usage after stream completes
final = stream.get_final_message()
print(f"\nUsage: {final.usage.input_tokens} in"
      f" / {final.usage.output_tokens} out")
```

Thực hành

Live Demo & Lab

05

1. Intent-driven

Nói rõ mục tiêu, output mong muốn, tiêu chí thành công.

2. Context-first

Đưa đủ file, ví dụ, ràng buộc, lỗi gặp phải.

3. Human review

AI viết nhanh; con người chịu trách nhiệm chốt.

Từ “mình gõ code”

→ sang “mình đạo diễn AI viết code”.

Tool stack gợi ý:

Cursor / VS Code + Copilot hoặc Claude Code.

Prompt coding tốt = mục tiêu rõ + context đủ + pattern cần theo + cách test.

1. Mở terminal, tạo project: `mkdir chatbot && cd chatbot`
2. Tạo virtual env: `python -m venv .venv && source .venv/bin/activate`
3. Cài packages: `pip install anthropic openai`
4. Set API key: `export ANTHROPIC_API_KEY="sk-ant- ..."`
5. Viết `chatbot.py` – gọi API + streaming
6. Chạy và test với 3 prompts khác nhau
7. Đo latency: `time.time()` trước/sau API call

Mục tiêu: Gọi API thực tế: so sánh 3 providers về latency, cost, quality

Deliverable: Script Python hoàn chỉnh: gọi Anthropic + OpenAI + Gemini, chatbot có streaming, bảng so sánh kết quả

Thời gian: 45 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 LLM = Transformer dự đoán token tiếp theo từ context
- 2 Để usable, LLM đi qua Pre-training → SFT → alignment
- 3 Chọn model theo trade-off quality, latency, cost
- 4 Một API call luôn có prompt, response, usage và stop reason
- 5 Vibe Coding tốt = intent rõ + context đủ + review kỹ

Ngày 2: Prompt Engineering

*“Prompt tốt tạo ra output tốt.
Nhưng “tốt” nghĩa là gì?”*

- Hoàn thành Lab #1
- Đọc: “Attention Is All You Need” (2017)
- Thử gọi API với 3 prompts khác nhau

1. Hochreiter & Schmidhuber (1997). “Long Short-Term Memory”. *Neural Computation* 9(8)
2. Bahdanau, Cho & Bengio (2014). “Neural Machine Translation by Jointly Learning to Align and Translate”. arXiv:1409.0473
3. Vaswani et al. (2017). “Attention Is All You Need”. arXiv:1706.03762
4. Devlin et al. (2018). “BERT: Pre-training of Deep Bidirectional Transformers”. arXiv:1810.04805
5. Ouyang et al. (2022). “InstructGPT / RLHF”. arXiv:2203.02155
6. Rafailov et al. (2023). “DPO”. arXiv:2305.18290
7. Karpathy (2023). “State of GPT” practitioner talk
8. Anthropic / OpenAI / Google API quickstarts

Hỏi & Đáp

Bạn có câu hỏi nào về LLM, Transformer, Token Economy, hoặc API?



Cảm ơn!

Phạm Mạnh

Email: a.nguyen@vinuni.edu.vn

Tài liệu: github.com/vinuni/aicb-materials

Bài tập nộp trước Ngày 2 · Đọc thêm: Vaswani et al. (2017)